Telco Churn Analysis

Import necessary libraries & load data

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path

df = pd.read_csv("telco-customer-churn.csv")
```

Initial Data Exploration

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
```

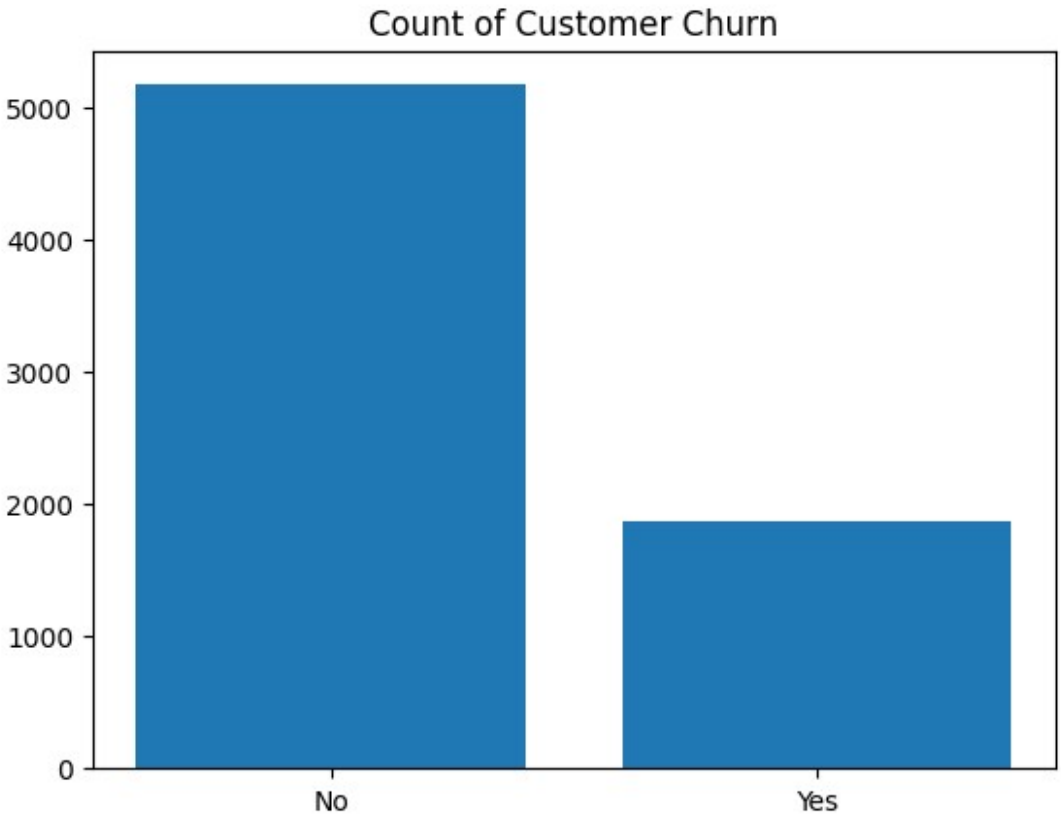
```
df.head(4)
```

```
   customerID  gender  SeniorCitizen Partner Dependents  tenure
PhoneService  \
0  7590-VHVEG  Female              0     Yes         No       1
No
1  5575-GNVDE    Male              0      No         No      34
Yes
2  3668-QPYBK    Male              0      No         No       2
Yes
3  7795-CFOCW    Male              0      No         No      45
No

      MultipleLines InternetService OnlineSecurity  ...
DeviceProtection  \
0  No phone service             DSL             No  ...
No
1                No             DSL            Yes  ...
Yes
2                No             DSL            Yes  ...
No
3  No phone service             DSL            Yes  ...
Yes

  TechSupport StreamingTV StreamingMovies        Contract
PaperlessBilling  \
0          No          No              No  Month-to-month
Yes
1          No          No              No        One year
No
2          No          No              No  Month-to-month
Yes
3         Yes          No              No        One year
No

               PaymentMethod MonthlyCharges  TotalCharges Churn
0           Electronic check          29.85         29.85    No
1               Mailed check          56.95        1889.5    No
2               Mailed check          53.85        108.15   Yes
3  Bank transfer (automatic)          42.30       1840.75    No

[4 rows x 21 columns]
```

Initial Data Exploration

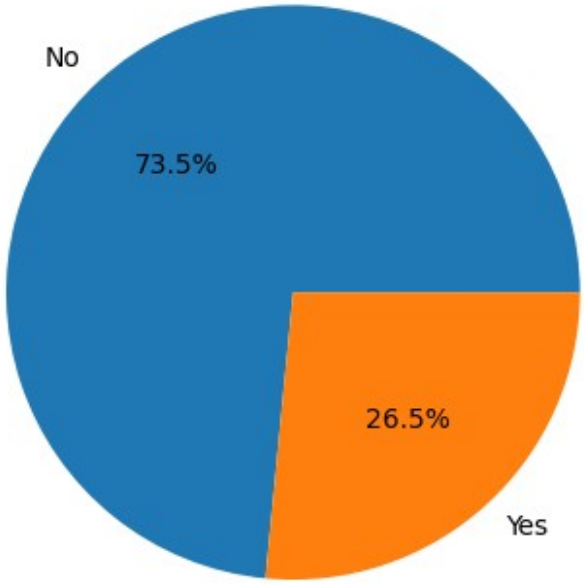
```
churn = df['Churn'].value_counts()
plt.title('Count of Customer Churn')
plt.bar(churn.index, churn.values)
```

```
<BarContainer object of 2 artists>
```




```
pct_churn = df['Churn'].value_counts(normalize=True)
plt.pie(pct_churn, labels=pct_churn.index, autopct='%1.1f%%')
```

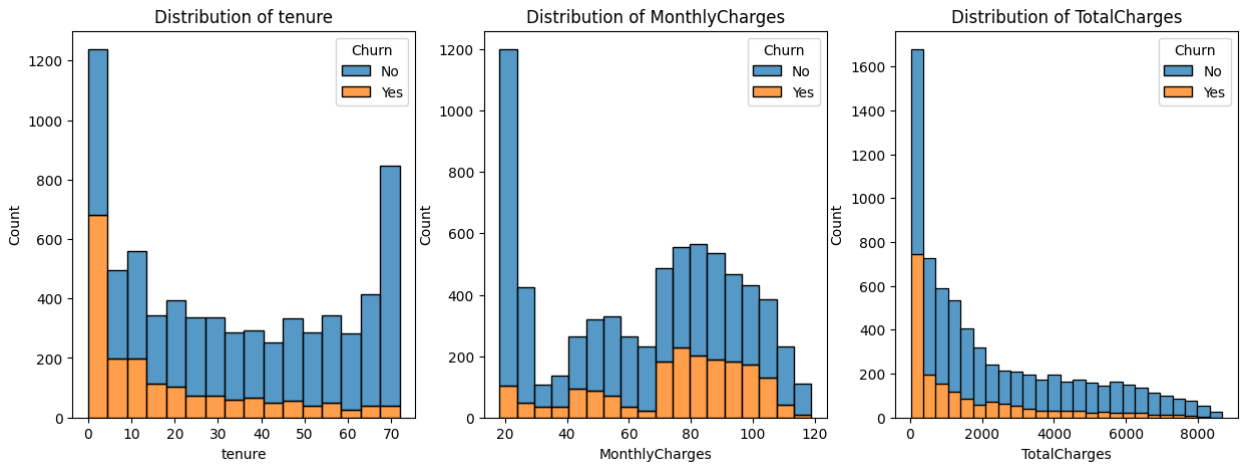
```
([<matplotlib.patches.Wedge at 0x114e59767b0>,
  <matplotlib.patches.Wedge at 0x114e5a1b4d0>],
 [Text(-0.7393678155529122, 0.8144539479458092, 'No'),
  Text(0.7393680809356542, -0.8144537070291521, 'Yes')],
 [Text(-0.40329153575613386, 0.4442476079704413, '73.5%'),
  Text(0.4032916805103568, -0.44424747656135566, '26.5%')])
```



Understanding the data that causes churn

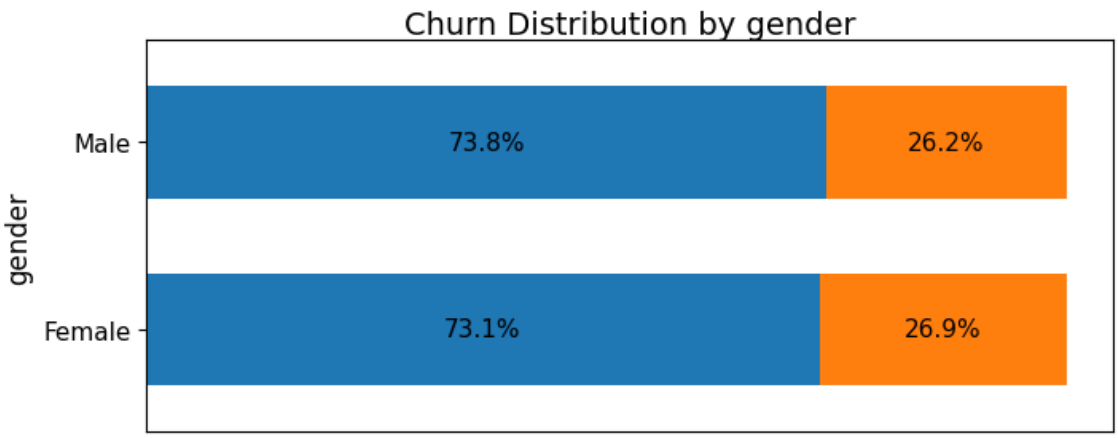
Numeric Features

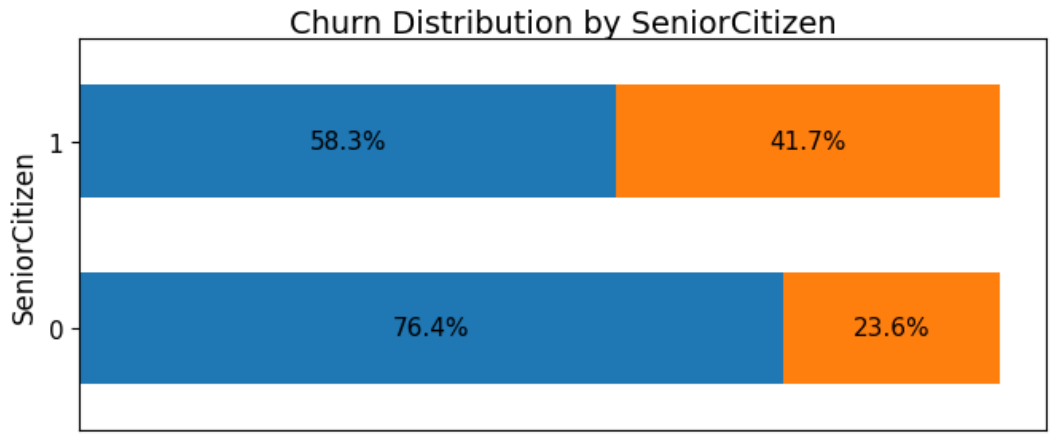
```
#| column: body-outset-left
#| out-width: 100%
#| fig-align: center
#| width: 100%
#| fig-caption: Distribution of numerical features
numerical_features = ['tenure', 'MonthlyCharges', 'TotalCharges']
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
for i, feature in enumerate(numerical_features):
    if feature == 'TotalCharges':
        df[feature] = pd.to_numeric(df[feature], errors='coerce')  #
Convert to numeric
    sns.histplot(data=df, x=feature, hue='Churn', multiple="stack",
ax=axes[i])
    axes[i].set_title(f'Distribution of {feature}')
```

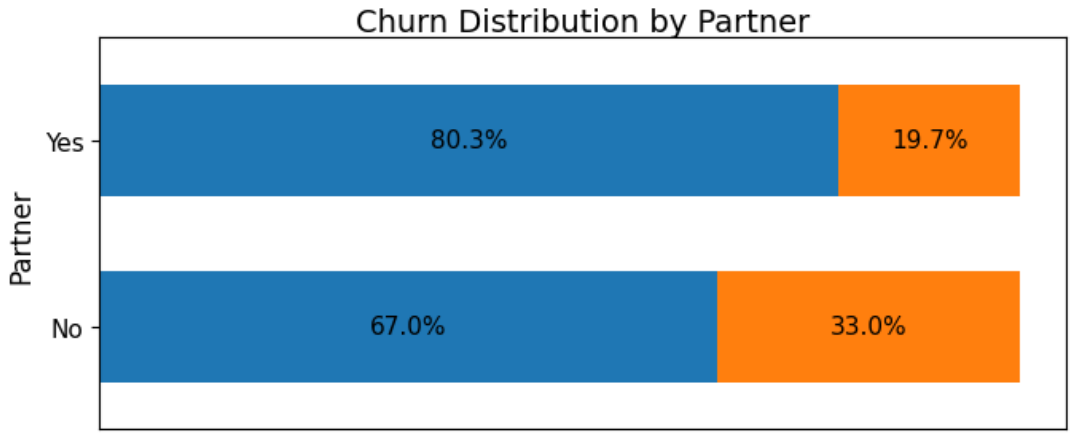


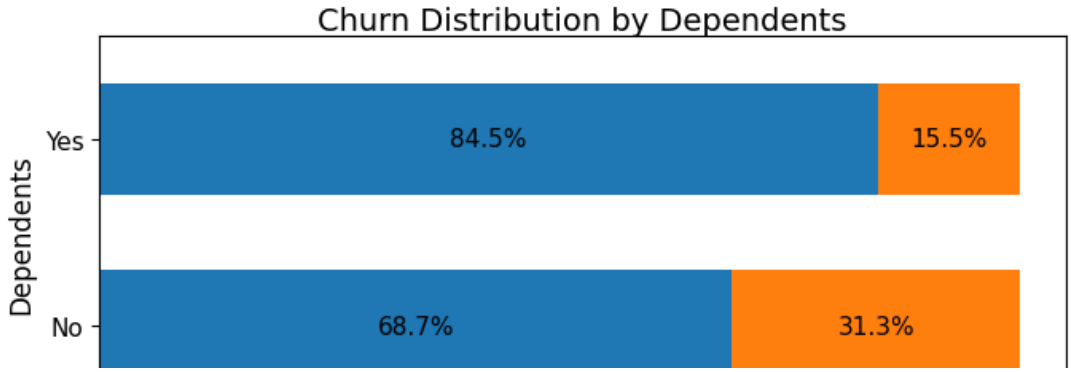

```
categorical_features = ['gender', 'SeniorCitizen', 'Partner',
'Dependents', 'PhoneService', 'InternetService', 'Contract']
fig, axes = plt.subplots(7, 1, figsize=(8, 20))
axes = axes.flatten()

plt.rcParams.update({'font.size': 12})  # Increase base font size

for i, feature in enumerate(categorical_features):
    # Calculate percentages
    percentages = (df.groupby(feature)['Churn']
                    .value_counts(normalize=True)
                    .unstack()
                    .mul(100))

    # Create horizontal stacked bars
    percentages.plot(kind='barh',
                    stacked=True,
                    ax=axes[i],
                    legend=False,
                    width=0.6)  # Changed from height to width

    # Customize the plot
    axes[i].set_title(f'Churn Distribution by {feature}', fontsize=14,
pad=-30)
    axes[i].set_ylabel(feature, fontsize=12)

    # Add percentage labels on the bars
    for c in axes[i].containers:
        axes[i].bar_label(c, fmt='%.1f%%', label_type='center',
fontsize=11)

    # Remove x-axis percentage labels
    axes[i].set_xticks([])

    # Add border around the subplot
```

```
    for spine in axes[i].spines.values():
        spine.set_visible(True)

    # Make tick labels larger
    axes[i].tick_params(axis='both', which='major', labelsize=11)

    # Adjust plot to reduce white space
    axes[i].margins(y=0.15)  # Reduce vertical margins

# Remove empty subplots
for j in range(i+1, len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
plt.show()
```


Churn Distribution by gender
gender
Male
Female
73.8%
26.2%
73.1%
26.9%

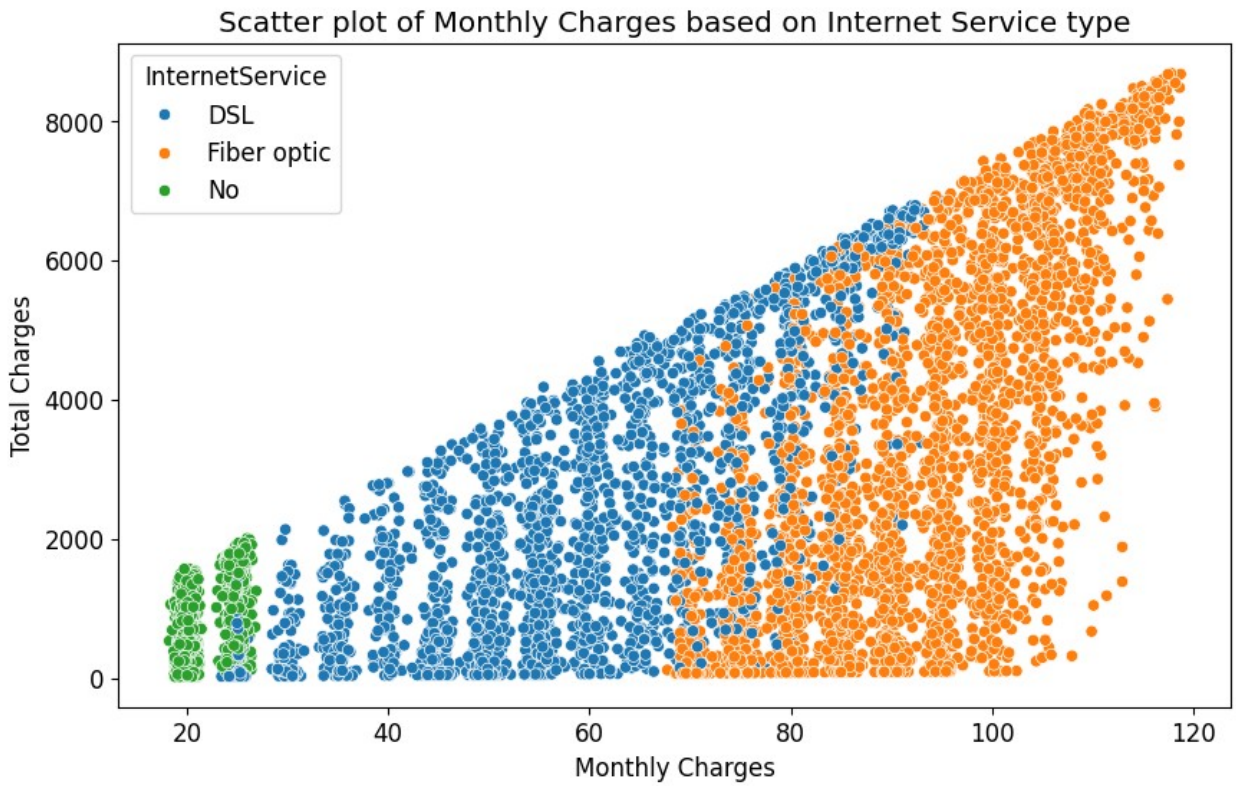

Churn Distribution by SeniorCitizen
SeniorCitizen
1
0
58.3%
41.7%
76.4%
23.6%


Churn Distribution by Partner
Partner
Yes
No
80.3%
19.7%
67.0%
33.0%

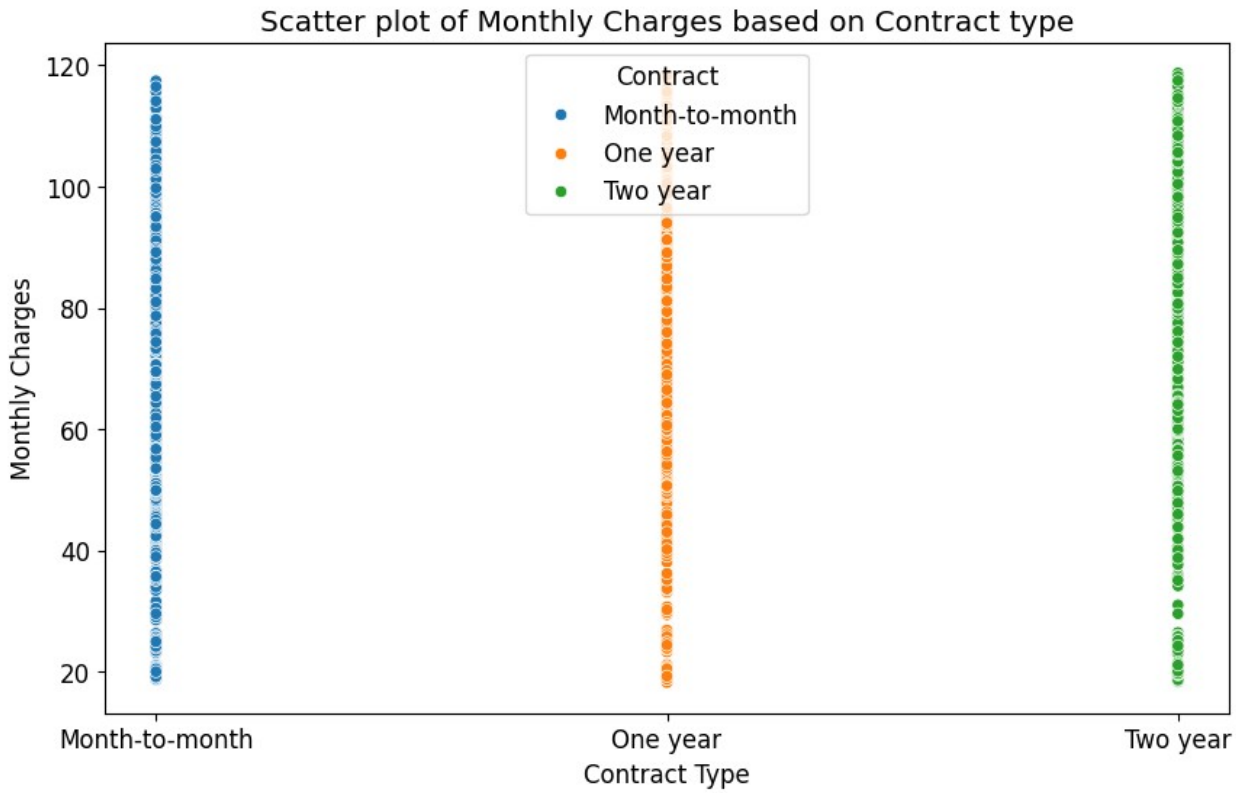

Churn Distribution by Dependents
Dependents
Yes
No
84.5%
15.5%
68.7%
31.3%

Some more detailed analysis

```
plt.figure(figsize=(10, 6))
sns.scatterplot(data=df, x='MonthlyCharges', y='TotalCharges',
hue='InternetService')
plt.title('Scatter plot of Monthly Charges based on Internet Service
type')
plt.xlabel('Monthly Charges')
plt.ylabel('Total Charges')
plt.show()
```



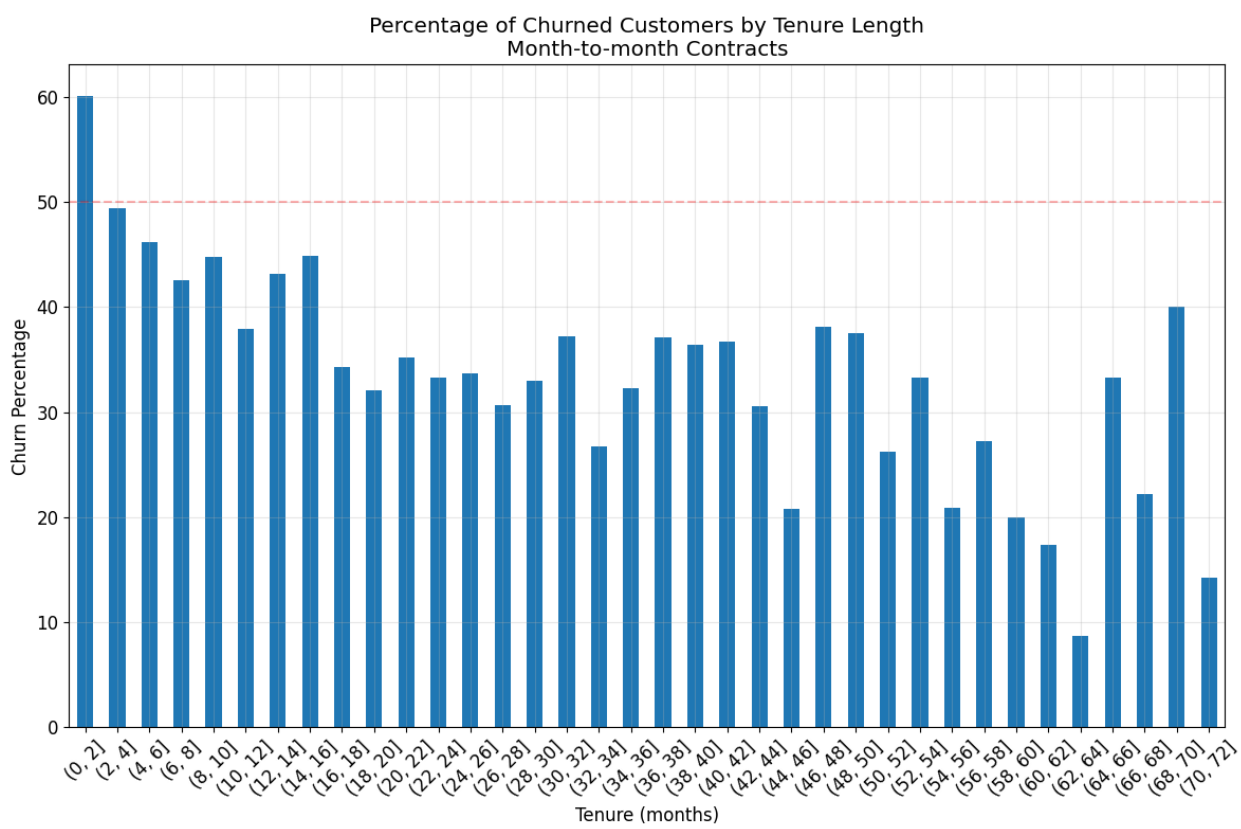

```
plt.figure(figsize=(10, 6))
sns.scatterplot(data=df, x='Contract', y='MonthlyCharges',
hue='Contract')
plt.title('Scatter plot of Monthly Charges based on Contract type')
plt.xlabel('Contract Type')
plt.ylabel('Monthly Charges')
plt.show()
```




```
def plot_churn_by_tenure(data, contract_type):
    # Create the bins
    bins = np.arange(0, data['tenure'].max() + 2, 2)  # +2 to include
the last value
    data['tenure_bin'] = pd.cut(data['tenure'], bins=bins)

    # Calculate percentage of churned customers in each bin
    churn_by_tenure = (data.groupby('tenure_bin')['Churn']
                        .value_counts(normalize=True)
                        .unstack())

    plt.figure(figsize=(12, 8))
    churn_by_tenure['Yes'].multiply(100).plot(kind='bar')
    plt.title(f'Percentage of Churned Customers by Tenure Length\
n{contract_type} Contracts')
    plt.xlabel('Tenure (months)')
    plt.ylabel('Churn Percentage')
    plt.axhline(y=50, color='r', linestyle='--', alpha=0.3)
    plt.grid(True, alpha=0.3)
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()

    # Print statistics
    # print(f"\nChurn percentage by tenure bins for {contract_type}
```

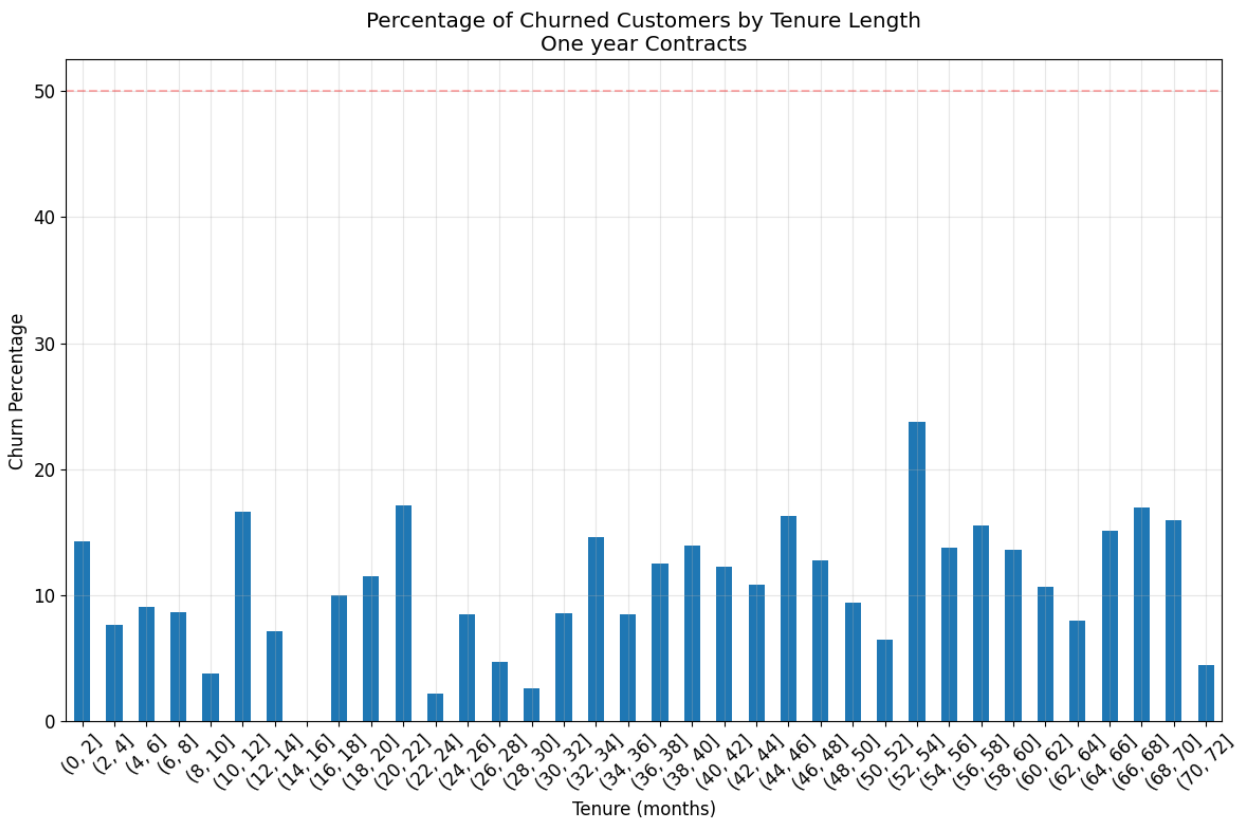
```
contracts:")
    # print(churn_by_tenure['Yes'].multiply(100).round(1))

# Create three dataframes
monthly = df[df['Contract'] == 'Month-to-month']
one_year = df[df['Contract'] == 'One year']
two_year = df[df['Contract'] == 'Two year']

plot_churn_by_tenure(monthly, 'Month-to-month')
```

```
C:\Users\samee\AppData\Local\Temp\ipykernel_3768\642956251.py:4:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
returning-a-view-versus-a-copy
  data['tenure_bin'] = pd.cut(data['tenure'], bins=bins)
C:\Users\samee\AppData\Local\Temp\ipykernel_3768\642956251.py:7:
FutureWarning: The default of observed=False is deprecated and will be
changed to True in a future version of pandas. Pass observed=False to
retain current behavior or observed=True to adopt the future default
and silence this warning.
  churn_by_tenure = (data.groupby('tenure_bin')['Churn']
```



```
plot_churn_by_tenure(one_year, 'One year')
```

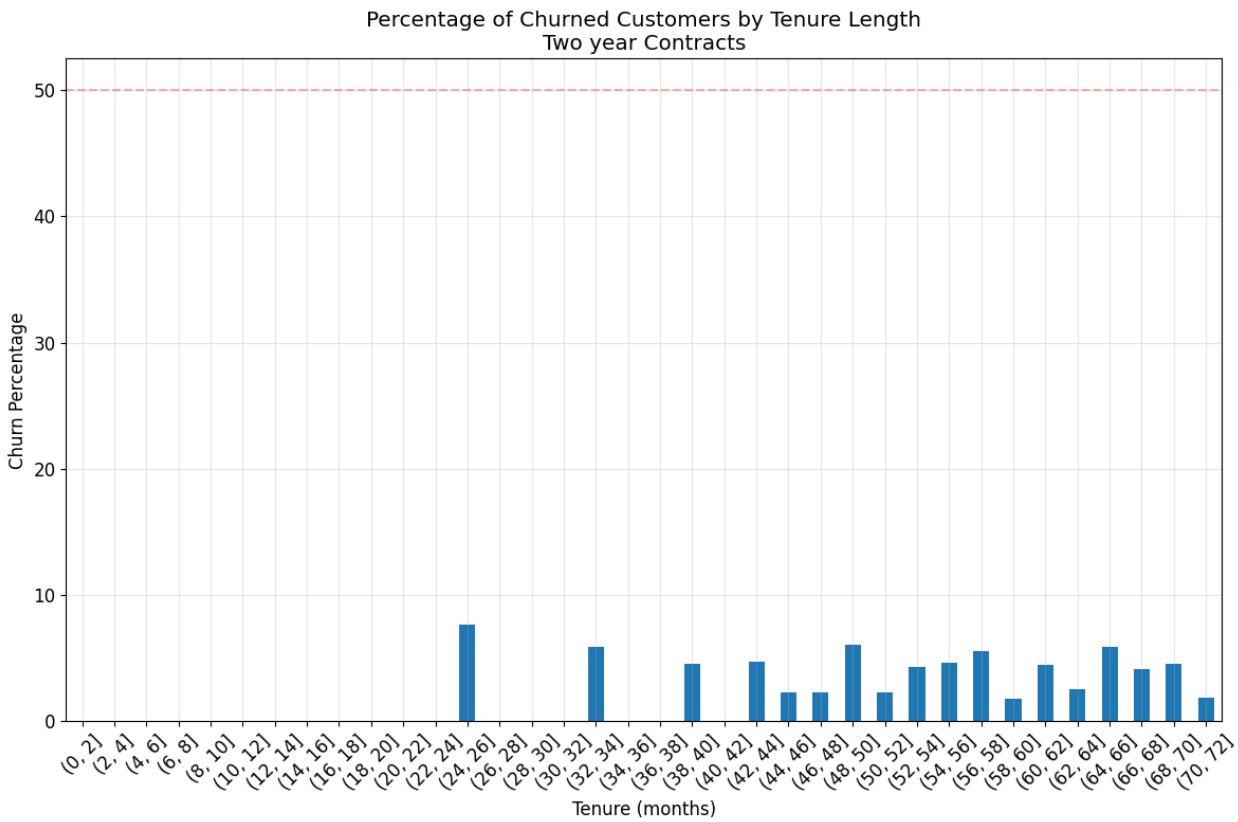
```
C:\Users\samee\AppData\Local\Temp\ipykernel_3768\642956251.py:4:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
returning-a-view-versus-a-copy
  data['tenure_bin'] = pd.cut(data['tenure'], bins=bins)
C:\Users\samee\AppData\Local\Temp\ipykernel_3768\642956251.py:7:
FutureWarning: The default of observed=False is deprecated and will be
changed to True in a future version of pandas. Pass observed=False to
retain current behavior or observed=True to adopt the future default
and silence this warning.
  churn_by_tenure = (data.groupby('tenure_bin')['Churn']
```




```
plot_churn_by_tenure(two_year, 'Two year')
```

```
C:\Users\samee\AppData\Local\Temp\ipykernel_3768\642956251.py:4:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

```
See the caveats in the documentation: 
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
returning-a-view-versus-a-copy
  data['tenure_bin'] = pd.cut(data['tenure'], bins=bins)
C:\Users\samee\AppData\Local\Temp\ipykernel_3768\642956251.py:7:
FutureWarning: The default of observed=False is deprecated and will be
changed to True in a future version of pandas. Pass observed=False to
retain current behavior or observed=True to adopt the future default
and silence this warning.
  churn_by_tenure = (data.groupby('tenure_bin')['Churn']
```



Model building

```
# Prep data

# Convert columns to binary and create new lowercase columns
df['gender_male'] = (df.gender=='Male')
df['senior_citizen'] = (df.SeniorCitizen==1)
df['partner'] = (df.Partner=='Yes')
df['dependents'] = (df.Dependents=='Yes')
df['phone_service'] = (df.PhoneService=='Yes')
df['multiple_lines'] = (df.MultipleLines=='Yes')
```

```
df['single_line'] = (df.MultipleLines=='No')

df['fiber_optic'] = (df.InternetService=='Fiber optic')
df['dsl'] = (df.InternetService=='DSL')

df['online_security'] = (df.OnlineSecurity=='Yes')
df['online_backup'] = (df.OnlineBackup=='Yes')
df['device_protection'] = (df.DeviceProtection=='Yes')
df['tech_support'] = (df.TechSupport=='Yes')
df['streaming_tv'] = (df.StreamingTV=='Yes')
df['streaming_movies'] = (df.StreamingMovies=='Yes')

df['month_to_month'] = (df.Contract=='Month-to-month')
df['one_year'] = (df.Contract=='One year')
df['two_year'] = (df.Contract=='Two year')

df['paperless_billing'] = (df.PaperlessBilling=='Yes')
df['electronic_check'] = (df.PaymentMethod=='Electronic check')
df['mailed_check'] = (df.PaymentMethod=='Mailed check')
df['bank_transfer'] = (df.PaymentMethod=='Bank transfer (automatic)')
df['credit_card'] = (df.PaymentMethod=='Credit card (automatic)')

df["total_charges"] = pd.to_numeric(df.TotalCharges,
errors='coerce').fillna(0)
df['monthly_charges'] = df.MonthlyCharges
df['tenure'] = df.tenure

df['churn'] = (df.Churn=='Yes')

# Drop all original columns
columns_to_drop = ['customerID', 'gender', 'SeniorCitizen', 'Partner',
'Dependents',
                   'PhoneService', 'MultipleLines', 'InternetService',
'OnlineSecurity',
                   'OnlineBackup', 'DeviceProtection', 'TechSupport',
'StreamingTV',
                   'StreamingMovies', 'Contract', 'PaperlessBilling',
'PaymentMethod',
                   'TotalCharges', 'MonthlyCharges', 'Churn']

df.drop(columns=columns_to_drop, inplace=True)
df.head()
```

```
   tenure  gender_male  senior_citizen  partner  dependents
phone_service  \
0       1        False           False     True       False
False
1      34         True           False    False       False
True
2       2         True           False    False       False
```

```
True
3      45         True           False    False       False
False
4       2        False           False    False       False
True

   multiple_lines  single_line  fiber_optic    dsl  ...  one_year
two_year  \
0           False        False        False   True  ...     False
False
1           False         True        False   True  ...      True
False
2           False         True        False   True  ...     False
False
3           False        False        False   True  ...      True
False
4           False         True         True  False  ...     False
False

   paperless_billing  electronic_check  mailed_check  bank_transfer  \
0               True              True         False          False
1              False             False          True          False
2               True             False          True          False
3              False             False         False           True
4               True              True         False          False

   credit_card  total_charges  monthly_charges  churn
0        False          29.85            29.85  False
1        False        1889.50            56.95  False
2        False         108.15            53.85   True
3        False        1840.75            42.30  False
4        False         151.65            70.70   True

[5 rows x 27 columns]
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 27 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   tenure             7043 non-null   int64
 1   gender_male        7043 non-null   bool
 2   senior_citizen     7043 non-null   bool
 3   partner            7043 non-null   bool
 4   dependents         7043 non-null   bool
 5   phone_service      7043 non-null   bool
 6   multiple_lines     7043 non-null   bool
 7   single_line        7043 non-null   bool
```

```
 8   fiber_optic         7043 non-null   bool
 9   dsl                 7043 non-null   bool
 10  online_security     7043 non-null   bool
 11  online_backup       7043 non-null   bool
 12  device_protection   7043 non-null   bool
 13  tech_support        7043 non-null   bool
 14  streaming_tv        7043 non-null   bool
 15  streaming_movies    7043 non-null   bool
 16  month_to_month      7043 non-null   bool
 17  one_year            7043 non-null   bool
 18  two_year            7043 non-null   bool
 19  paperless_billing   7043 non-null   bool
 20  electronic_check    7043 non-null   bool
 21  mailed_check        7043 non-null   bool
 22  bank_transfer       7043 non-null   bool
 23  credit_card         7043 non-null   bool
 24  total_charges       7043 non-null   float64
 25  monthly_charges     7043 non-null   float64
 26  churn               7043 non-null   bool
dtypes: bool(24), float64(2), int64(1)
memory usage: 330.3 KB
```

□ Always learning, always improving □

□ Project Code & Notebook:□

□ GitHub:[https://lnkd.in/eHH-Dbmy]

□ LinkedIn:[https://lnkd.in/gh67YnGy]

😊If you found this project interesting, feel free to like □, comment □, and share your feedback in the comments. It would really mean a lot!

□ Author

😊Sameer Ali

Aspiring Data Analysis | Python | Data Scientist